Reinforcement Learning

A Complete Guide — From the Bellman Equation to ROS 2 Robots and Humanoid Locomotion

Prepared for Sunkeerth

September 2026

Contents

1 Chapter 1 — Introduction: Why Reinforcement Learning Is a Different Kind of Machine Learning **2**
- 1.1 The One-Sentence Idea . . . 2
- 1.2 An Analogy: Learning to Ride a Bike vs. Reading a Manual . . . 2
- 1.3 Why This Is a Genuinely Different Problem From Supervised Learning . . . 2
- 1.4 The Agent-Environment Loop . . . 2
- 1.5 Core Vocabulary . . . 3
- 1.6 Three Structural Challenges That Make RL Hard . . . 3
- 1.7 A Five-Second History . . . 4
- 1.8 What This Book Covers, In Order . . . 4

2 Chapter 2 — Markov Decision Processes **5**
- 2.1 The Formal Object Every RL Algorithm Solves . . . 5
- 2.2 The Markov Property . . . 5
- 2.3 Episodic vs. Continuing Tasks . . . 6
- 2.4 Policies . . . 6
- 2.5 A Worked Example: The Warehouse Grid . . . 6
- 2.6 Why Formalize With an MDP At All? . . . 6

3 Chapter 3 — Returns, Value Functions, and the Bellman Equations **7**
- 3.1 The Return: Formalizing "Long-Term Success" . . . 7
- 3.2 Value Functions . . . 7
- 3.3 The Bellman Equations: Recursion Is Everything . . . 7
- 3.4 Optimality . . . 8
- 3.5 Why Target V or Q Rather Than Search Policies Directly? . . . 8
- 3.6 A Worked Numerical Example: Solving a Tiny MDP by Hand . . . 8

4 Chapter 4 — Dynamic Programming: Solving Known MDPs **9**
- 4.1 Policy Evaluation . . . 9
- 4.2 Policy Improvement and Policy Iteration . . . 9
- 4.3 Value Iteration . . . 9
- 4.4 Why Dynamic Programming Doesn't Scale to Robots . . . 10

5 Chapter 5 — Monte Carlo Methods **11**
5.1 First-Visit vs. Every-Visit MC . . . 11
5.2 Incremental Updates . . . 11
5.3 Monte Carlo Control . . . 11
5.4 Strengths, Weaknesses, and When to Use MC . . . 12

6 Chapter 6 — Temporal-Difference (TD) Learning **13**
6.1 TD(0) . . . 13
6.2 Bias-Variance Trade-off: MC vs. TD . . . 13
6.3 A Worked Numerical Example: TD(0) vs. Monte Carlo, Step by Step . . . 14
6.4 n-step TD: A Spectrum, Not a Binary Choice . . . 14

7 Chapter 7 — SARSA and Q-Learning: Learning to Control **15**
7.1 SARSA: On-Policy TD Control . . . 15
7.2 Q-Learning: Off-Policy TD Control . . . 15
7.3 Cliff Walking, Worked . . . 15
7.4 Which Should You Use on a Real Robot? . . . 16
7.5 Expected SARSA: A Unifying View . . . 16

8 Chapter 8 — Eligibility Traces and TD(λ) **17**
8.1 The Eligibility Trace . . . 17
8.2 TD(λ): Distributing the TD-Error Backward . . . 17
8.3 SARSA(λ) and Practical Notes . . . 17
8.4 Why Introduce λ At All? . . . 18

9 Chapter 9 — Function Approximation **19**
9.1 Generalization: The Whole Point . . . 19
9.2 Gradient-Based Value Function Updates . . . 19
9.3 The Deadly Triad . . . 19
9.4 A Worked Example: How Generalization Actually Helps — and Hurts . . . 20

10 Chapter 10 — Deep Q-Networks (DQN) and Its Descendants **21**
10.1 The Loss Function . . . 21
10.2 Two Key Stabilization Tricks . . . 21
10.3 Notable Extensions . . . 22
10.4 The Core Limitation: Discrete Actions Only . . . 22
10.5 A Worked Example: DQN on a Tiny Problem . . . 22

11 Chapter 11 — Policy Gradient Methods: REINFORCE **23**
11.1 The Policy Gradient Theorem . . . 23
11.2 The Variance Problem, and Baselines . . . 23
11.3 A Worked Example . . . 23

12 Chapter 12 — Actor-Critic Methods **24**
12.1 The Actor-Critic Update . . . 24
12.2 The Advantage Function . . . 24
12.3 A2C and A3C . . . 25

13 Chapter 13 — Trust Regions and Proximal Policy Optimization (PPO) **26**

13.1 The Probability Ratio . . . 26
13.2 The Clipped Surrogate Objective . . . 26
13.3 A Worked Numerical Example . . . 26
13.4 Generalized Advantage Estimation (GAE) . . . 27
13.5 Why PPO Became the Default for Applied RL . . . 27
13.6 Practical Hyperparameters . . . 27

14 Chapter 14 — Continuous Control: DDPG, TD3, and SAC 28
14.1 DDPG: Deep Deterministic Policy Gradient . . . 28
14.2 TD3: Fixing DDPG's Overestimation . . . 28
14.3 SAC: Soft Actor-Critic . . . 29
14.4 Choosing Among PPO, DDPG/TD3, and SAC for Robotics . . . 29

15 Chapter 15 — Exploration, Model-Based RL, and Multi-Agent Systems 30
15.1 Exploration Strategies . . . 30
15.2 Model-Based Reinforcement Learning . . . 30
15.3 Multi-Agent RL . . . 31
15.4 Reward Shaping and Its Pitfalls . . . 31

16 Chapter 16 — Reinforcement Learning in Robotics, ROS 2, and Embodied AI 32
16.1 The Core Idea: A ROS 2 Node Is Just Another RL Environment . . . 32
16.2 The Gym-Style Wrapper . . . 32
16.3 Training vs. Deployment . . . 33
16.4 The Sim-to-Real Gap and Domain Randomization . . . 33
16.5 Reward Design for AMR Navigation . . . 33
16.6 RL vs. Classical Control: An Architecture Decision . . . 34
16.7 RL for Legged and Humanoid Locomotion . . . 34
16.8 Why This Matters for ROS 2-Based Systems Specifically . . . 35

17 Chapter 17 — Implementing Reinforcement Learning in Python 36
17.1 Tabular Q-Learning (Chapter 7) — From Scratch . . . 36
17.2 Value Iteration (Chapter 4) — From Scratch . . . 36
17.3 A Minimal DQN Training Step (Chapter 10) . . . 36
17.4 PPO via Stable-Baselines3 (Chapter 13) — What You'll Actually Do Most Often . . . 37
17.5 A ROS 2 / Gym-Style Environment Wrapper (Chapter 16) . . . 37
17.6 Deploying a Trained Policy as a ROS 2 Node . . . 38
17.7 Closing Note . . . 38

1 Chapter 1 — Introduction: Why Reinforcement Learning Is a Different Kind of Machine Learning

1.1 The One-Sentence Idea

Reinforcement Learning (RL) is the branch of machine learning built around one core setup: an **agent** takes actions inside an **environment**, receives a **reward** signal after each action, and learns — purely through trial and error — a **policy** that maximizes cumulative reward over time. There is no dataset of correct answers handed to it upfront; the agent has to discover what works by acting and observing the consequences.

1.2 An Analogy: Learning to Ride a Bike vs. Reading a Manual

Imagine trying to learn to ride a bicycle by reading a manual that lists, for every possible combination of speed, lean angle, and steering position, the "correct" handlebar adjustment. Nobody learns this way, and no such manual could realistically be written — the combinations are endless, and many situations (a sudden gust of wind, a pebble under the front wheel) are never anticipated in advance. Instead, you get on the bike, wobble, fall, adjust, and slowly discover — through the feedback of staying upright or falling — what to do. Reinforcement learning formalizes exactly this loop: try something, observe the consequence (reward), and adjust behavior accordingly, with no manual required.

1.3 Why This Is a Genuinely Different Problem From Supervised Learning

Property	**Supervised Learning**	**Reinforcement Learning**
Training signal	Correct label given for every example	A scalar reward, often delayed and sparse
Data source	Fixed dataset, collected once	Generated by the agent's own actions
Feedback timing	Immediate, per example	Can arrive many steps after the action that caused it
Distribution	Fixed (i.i.d. assumption)	Changes as the policy itself changes

This last row is worth dwelling on: because the agent's own behavior determines what data it sees next, and that behavior is constantly changing as learning proceeds, RL breaks the fixed-distribution assumption that most of supervised learning relies on. This single structural difference is the reason RL needs its own family of algorithms rather than simply reusing supervised learning techniques.

1.4 The Agent-Environment Loop

Every RL problem is described by the same loop, repeated at every discrete timestep t: the agent observes a state S_t, selects an action A_t according to its policy, and the environment responds with a new state S_{t+1} and a reward R_{t+1}.

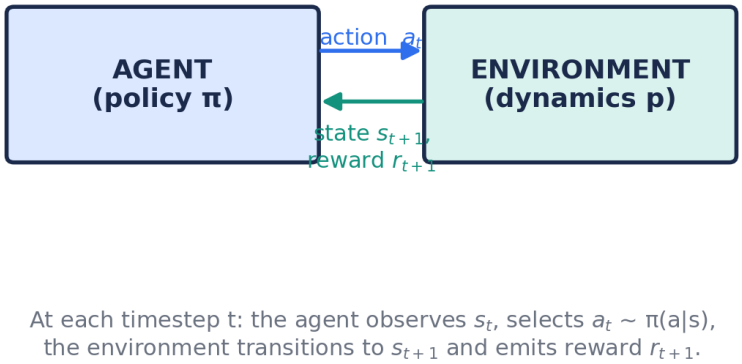


Figure 1: The agent-environment interaction loop underlying every algorithm in this book.

1.5 Core Vocabulary

- **State (s):** a description of the situation the agent is in — e.g. a robot's LiDAR scan, joint angles, or a camera image.
- **Action (a):** a choice the agent can make — e.g. set wheel velocities, apply a joint torque.
- **Reward (r):** a scalar feedback signal after each action, defining the goal implicitly.
- **Policy (π):** the agent's behavior — a mapping from states to actions, or to a probability distribution over actions.
- **Value function (V or Q):** a prediction of how much cumulative future reward can be expected from a state (or state-action pair).
- **Model:** the environment's transition dynamics $p(s' \mid s, a)$ and reward function — known in *model-based* settings, unknown in *model-free* settings (Chapter 4 vs. Chapters 5–14).
- **Episode:** one complete trajectory from a start state to a terminal state.

1.6 Three Structural Challenges That Make RL Hard

Credit assignment. When a robot collides with an obstacle at timestep 40, which of the previous 39 actions actually caused it? RL algorithms must apportion credit or blame across time — the entire subject of Chapters 3, 6, and 8.

Exploration vs. exploitation. An agent must balance exploiting what it already believes is the best action against exploring alternatives that might be better but are unproven. Too little exploration and the agent gets stuck in a mediocre habit; too much and it never converges (Chapter 15 returns to this in depth).

Non-stationarity from the agent's own learning. Because the policy is changing as the agent learns, the distribution of states and actions it experiences is a moving target, unlike the

fixed training distribution of supervised learning. This is a running theme through Chapters 9–14, where it interacts directly with function approximation.

1.7 A Five-Second History

- **1980s–1990s:** Dynamic programming and temporal-difference learning (Chapters 4 and 6) are formalized; Q-learning (Watkins, 1989) and TD-Gammon (Tesauro, 1992) demonstrate the core ideas work.
- **2013–2015:** Deep Q-Networks (DQN, Chapter 10) combine Q-learning with deep neural networks, learning to play Atari games directly from pixels — the beginning of *deep* RL.
- **2015–2017:** Policy gradient and actor-critic methods mature (Chapters 11–12); Proximal Policy Optimization (PPO, Chapter 13) becomes a robust, widely-adopted default.
- **2018–present:** Soft Actor-Critic (SAC) and other continuous-control methods (Chapter 14) make RL practical for real robot control; RL increasingly appears as a component layered on top of, rather than a replacement for, classical robotics engineering (Chapters 15–16).

1.8 What This Book Covers, In Order

Every layer of the problem, in the order you actually need it: the Markov Decision Process formalism (Ch. 2) → value functions and the Bellman equation (Ch. 3) → exact solutions via dynamic programming (Ch. 4) → learning from sampled experience via Monte Carlo (Ch. 5) and temporal-difference methods (Ch. 6) → control algorithms SARSA and Q-learning (Ch. 7) → eligibility traces (Ch. 8) → function approximation (Ch. 9) → Deep Q-Networks (Ch. 10) → policy gradients (Ch. 11) → actor-critic methods (Ch. 12) → PPO (Ch. 13) → continuous control: DDPG, TD3, SAC (Ch. 14) → exploration, model-based, and multi-agent RL (Ch. 15) → RL in robotics, ROS 2, and embodied AI (Ch. 16) → and finally working PyTorch/Stable-Baselines3 code for everything (Ch. 17).

2 Chapter 2 — Markov Decision Processes

2.1 The Formal Object Every RL Algorithm Solves

The Markov Decision Process (MDP) is the mathematical formalism underlying essentially all of reinforcement learning. An MDP is the tuple (S, A, P, R, γ):

- S — the set of all possible states
- A — the set of all possible actions
- $P(s' \mid s, a)$ — the transition probability function (the environment's dynamics)
- $R(s, a, s')$ — the expected reward function
- $\gamma \in [0, 1]$ — the discount factor, controlling how much future reward matters relative to immediate reward

2.2 The Markov Property

A state S_t is *Markov* if it captures all information relevant to the future:

$$P(S_{t+1} \mid S_t, A_t) = P(S_{t+1} \mid S_1, A_1, \ldots, S_t, A_t).$$

In plain terms: the future depends only on the present state and action, not on the full history that led there. A single LiDAR scan is *not* Markov for navigation (you don't know velocity from one snapshot), but a scan concatenated with the last commanded velocity restores the property approximately — exactly the observation-design problem revisited in Chapter 16.

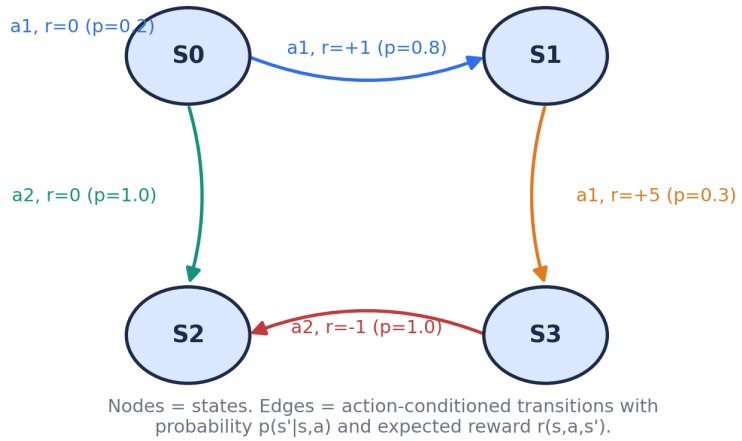


Figure 2: An MDP as a directed graph: states as nodes, action-labeled edges carrying transition probabilities and expected rewards.

2.3 Episodic vs. Continuing Tasks

Property	Episodic task	Continuing task
Termination	Reaches a terminal state (goal / collision / timeout)	Runs indefinitely
Example	One AMR delivery run, dock to bay	A factory robot that never "finishes"
Return	Finite sum of rewards to episode end	Requires $\gamma < 1$ to keep the sum finite

2.4 Policies

A policy π defines the agent's behavior. It can be deterministic, $\pi(s) = a$, or stochastic, $\pi(a \mid s) =$ probability of choosing a in state s. Stochastic policies are essential during learning (they enable exploration) and are the natural output of most deep RL algorithms (Chapters 11–14) — e.g. a Gaussian distribution over continuous wheel velocities.

2.5 A Worked Example: The Warehouse Grid

Let $S = \{25 \text{ grid cells}\}$, $A = \{\text{North, South, East, West}\}$, and $R = -1$ per step (encouraging speed) with $+100$ for reaching the drop-off cell. P is deterministic except for a 10% chance of "wheel slip" moving the robot to a random adjacent cell instead of the intended one. This tiny MDP is exactly solvable with the dynamic-programming methods of Chapter 4, and is structurally identical to a discretized AMR navigation problem (Chapter 16).

2.6 Why Formalize With an MDP At All?

Because the MDP framing separates *what the task is* (S, A, R) from *how to solve it* (the algorithm). Once a problem is written as an MDP, dozens of well-understood algorithms become available — the same reason control engineers formalize a system as state-space equations before choosing between PID, LQR, or MPC.

3 Chapter 3 — Returns, Value Functions, and the Bellman Equations

3.1 The Return: Formalizing "Long-Term Success"

The return G_t is the total discounted reward the agent collects from time t onward:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

γ controls the effective planning horizon: near 0 makes the agent myopic; near 1 makes it far-sighted.

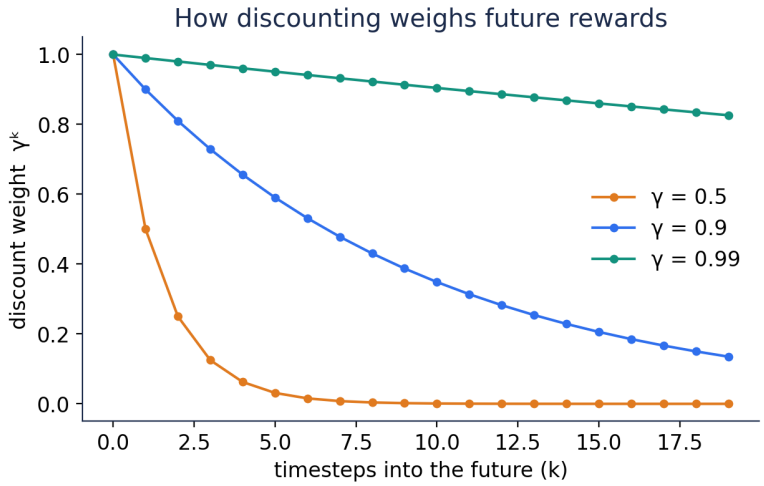


Figure 3: How much a reward k steps in the future is worth today, for three discount factors.

γ is like an interest rate in reverse: a reward 20 steps from now is worth γ^{20} times less than the same reward right now. A robot with $\gamma = 0.99$ is patient and takes a longer but safer path; one with $\gamma = 0.5$ only cares about the next couple of moves.

3.2 Value Functions

The state-value function is the expected return starting in state s and following π thereafter:

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]\,.$$

The action-value (Q) function is the expected return starting in s, taking action a, then following π:

$$Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]\,.$$

Q is more directly useful for control: comparing $Q(s,a_1)$ against $Q(s,a_2)$ immediately tells you which action is better, with no separate model required.

3.3 The Bellman Equations: Recursion Is Everything

The single most important idea in RL: value functions satisfy a recursive relationship — the value of a state equals the immediate reward plus the discounted value of wherever you end up next.

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\,\big[r + \gamma V^\pi(s')\big]$$

$$Q^{\pi}(s,a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \sum_{a'} \pi(a' \mid s') Q^{\pi}(s', a') \right]$$

For the optimal policy π^*, the Bellman *Optimality* Equation replaces the expectation over actions with a max:

$$V^*(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V^*(s') \right].$$

Planning a road trip by asking, at every city, "what's the best possible remaining trip from here?" rather than re-planning the whole journey from scratch is exactly this idea in equation form. Nearly every algorithm in this book — from tabular Q-learning to PPO — is, underneath, a different numerical method for solving this one recursive equation approximately.

3.4 Optimality

An optimal policy π^* achieves $V^{\pi^*}(s) \geq V^{\pi}(s)$ for every state and every other policy. A remarkable fact: there always exists at least one *deterministic* optimal policy for any finite MDP. Once V^* or Q^* is known, $\pi^*(s) = \arg\max_a Q^*(s,a)$.

3.5 Why Target V or Q Rather Than Search Policies Directly?

Because value functions satisfy a Bellman recursion that lets you compute (or approximate) them using only local, one-step information, refined repeatedly — dramatically more sample-efficient than evaluating every complete policy. The exception is the direct policy-search family of Chapters 11–14, which trades this efficiency for the ability to represent continuous, high-dimensional action spaces naturally — exactly what a robot's continuous velocity or torque commands need.

3.6 A Worked Numerical Example: Solving a Tiny MDP by Hand

Three states in a line: S1 – S2 – S3 (goal). Two actions in S1 and S2: *Right* (toward the goal) and *Stay*. Moving Right always succeeds. Reward is -1 per action in S1 or S2; the episode ends at S3. $\gamma = 0.9$.

Iteration	$V(S1)$	$V(S2)$	$V(S3)$	Note
0 (init)	0.00	0.00	0.00	—
1	-1.00	-1.00	0.00	first exposure to the immediate cost
2	-1.90	-1.00	0.00	S2's "Right" now sees S3's value
3	-1.90	-1.00	0.00	converged

Reading off the greedy policy: $\pi^*(S1) =$ Right, $\pi^*(S2) =$ Right — the obviously-optimal policy, now derived mechanically from the Bellman recursion rather than assumed. This is the single most useful debugging skill for later chapters: nearly every deep-RL bug traces back to a broken version of exactly this recursion.

4 Chapter 4 — Dynamic Programming: Solving Known MDPs

Dynamic Programming (DP) methods assume the MDP's model — P and R — is fully known. They rarely apply to real robots directly, but they are the conceptual foundation for every model-free method in Chapters 5–8, and they are exactly solvable, giving ground truth to sanity-check later approximate methods against.

4.1 Policy Evaluation

Given a fixed policy π, policy evaluation computes V^π by repeatedly applying the Bellman equation as an update rule until values stop changing.

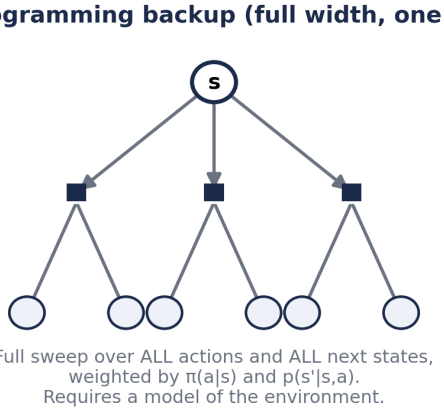


Figure 4: The DP backup diagram: a full sweep over every action and every possible next state, weighted by their probabilities.

4.2 Policy Improvement and Policy Iteration

Once V^π is known, improve the policy by acting greedily: $\pi'(s) = \arg\max_a \sum p(s', r \mid s, a)[r + \gamma V^\pi(s')]$. The Policy Improvement Theorem guarantees π' is at least as good as π everywhere. Alternating evaluation and improvement — *Policy Iteration* — provably converges to π^* in a finite number of iterations.

4.3 Value Iteration

Value Iteration shortcuts this by combining evaluation and improvement into a single update, taking the max over actions at every sweep:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V_k(s') \right].$$

Repeated until V stops changing meaningfully; the optimal policy is then read off greedily with respect to the converged V^*. For the 25-cell warehouse grid of Chapter 2, this typically converges in well under 100 sweeps, in about 20 lines of NumPy.

Figure 5: Policy Iteration: alternate full policy evaluation (E) and greedy improvement (I).

4.4 Why Dynamic Programming Doesn't Scale to Robots

- **Curse of dimensionality:** a robot's state (joint angles, velocities, camera pixels) can have millions of dimensions — an exact table over all states is impossible to store.
- **Requires a perfect model:** real friction, sensor noise, and actuator dynamics are rarely known exactly.
- **Full sweeps are expensive:** each backup considers every possible next state — infeasible for continuous state spaces.

Chapters 5–8 are best understood as *sample-based approximations to Dynamic Programming* — they compute the same Bellman backups, just from experienced trajectories instead of known probabilities, and typically for one state at a time instead of a full sweep. Understanding DP first makes every subsequent algorithm in this book look like a natural relaxation of it, not a disconnected trick.

5 Chapter 5 — Monte Carlo Methods

Monte Carlo (MC) methods learn value functions directly from experience — complete episodes — without requiring any knowledge of P or R. Run an episode to completion, observe the actual return G_t that followed each visited state, and average these returns over many episodes.

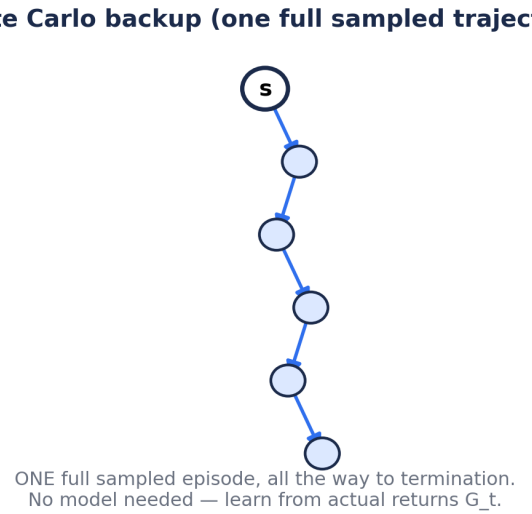


Figure 6: The Monte Carlo backup diagram: one full sampled trajectory, all the way to termination.

5.1 First-Visit vs. Every-Visit MC

If a state is visited multiple times within one episode, First-Visit MC only uses the return following the first occurrence; Every-Visit MC uses every occurrence. Both converge to V^π as episodes accumulate.

5.2 Incremental Updates

$$V(S_t) \leftarrow V(S_t) + \alpha\,[G_t - V(S_t)]$$

This is the template every value-learning update in this book follows: *new estimate* ← *old estimate* + *step size* × *(target − old estimate)*. The "target" changes between algorithms — here it's the actual observed return.

MC learning is like judging a chess opening purely by whether you won or lost the game it was part of — you wait for the whole game to finish, then credit every move with the final outcome. It's unbiased (the true final result) but noisy (a single game's outcome depends on many other moves and luck).

5.3 Monte Carlo Control

To find a good policy, MC Control alternates estimating $Q^\pi(s,a)$ from sampled episodes with improving π to be ε-greedy with respect to current estimates — a sample-based analogue of Policy Iteration.

5.4 Strengths, Weaknesses, and When to Use MC

Property	Monte Carlo
Bias	Unbiased — uses the true observed return
Variance	High — one episode's outcome depends on the whole trajectory of randomness
Needs a model?	No
Updates	Only at episode end
Continuing tasks	Not applicable — needs episodes to terminate
Good fit	Board games, short fixed-horizon tasks, offline evaluation of logged episodes

MC is a poor fit for most real-time robot control: you can't wait for an entire delivery run to finish before updating a navigation policy, and if the run ends in a collision, you'd like to learn from that immediately. This motivates Temporal-Difference learning, Chapter 6.

6 Chapter 6 — Temporal-Difference (TD) Learning

Temporal-Difference learning is, in Sutton and Barto's words, "the central and novel idea of reinforcement learning." It combines the best of Dynamic Programming and Monte Carlo: like MC, it learns directly from raw experience with no model; like DP, it bootstraps — updating estimates based partly on other learned estimates — without waiting for a final outcome.

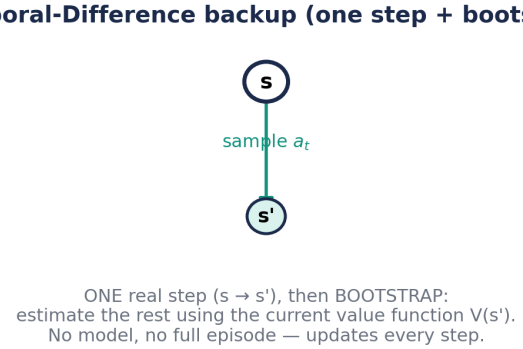


Figure 7: The TD backup diagram: one real step, then bootstrap off the current value estimate of the next state.

6.1 TD(0)

After every transition $(S_t, A_t, R_{t+1}, S_{t+1})$, TD(0) immediately updates:

$$V(S_t) \leftarrow V(S_t) + \alpha \left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \right].$$

The bracketed quantity is the **TD-error**, $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$, which reappears throughout this book, most notably driving actor-critic updates (Chapter 12).

TD learning is like adjusting your estimate of "how long is left on this road trip" at every town you pass, based on your current guess of how far the next landmark is — rather than waiting until you arrive home to find out you were wrong the whole way.

6.2 Bias-Variance Trade-off: MC vs. TD

	Monte Carlo	**TD(0)**
Bias	None (true return)	Some (bootstraps off a possibly-wrong estimate)
Variance	High	Low
Update frequency	End of episode	Every step
Continuing tasks	No	Yes

Why does almost every large-scale RL system (DQN, PPO, SAC — all of Chapters 10–14) build on TD ideas rather than pure Monte Carlo? Sample efficiency and applicability to long or continuing

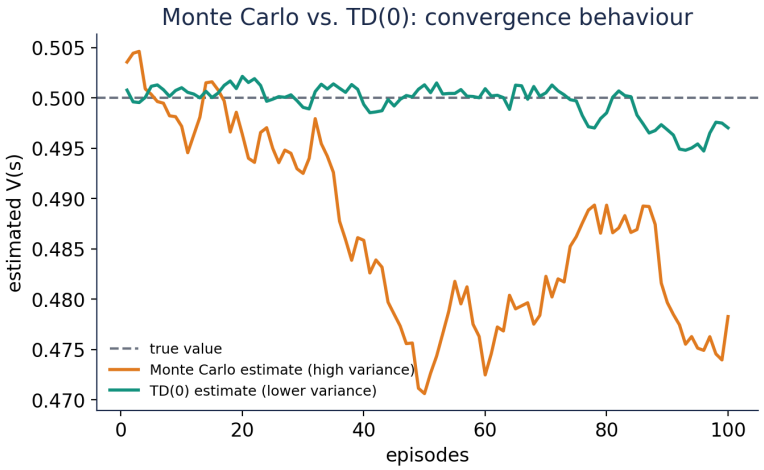


Figure 8: TD(0) typically converges with visibly lower variance than Monte Carlo.

tasks. A real robot cannot wait for full episodes before learning anything, and bootstrapping lets information propagate backward through the state space long before an episode ends.

6.3 A Worked Numerical Example: TD(0) vs. Monte Carlo, Step by Step

On the 3-cell corridor from Chapter 3, one episode: $S1 \to S2 \to S3$, rewards $-1, -1$, $\alpha = 0.5$, all estimates start at 0.

Monte Carlo (applied only at episode end): true return from S1 is $-1 + 0.9(-1) = -1.9$; from S2 is -1. Updates: $V(S1) \leftarrow -0.95$, $V(S2) \leftarrow -0.50$.

TD(0) (applied immediately, after each step): Step 1 uses $V(S2)$'s *current* value, still 0: $V(S1) \leftarrow 0 + 0.5[-1 + 0.9(0) - 0] = -0.50$. Step 2: $V(S2) \leftarrow -0.50$.

	$V(S1)$ after 1 episode	$V(S2)$ after 1 episode	Updated when
Monte Carlo	-0.95	-0.50	Only at episode end, using the true return
TD(0)	-0.50	-0.50	Immediately, using a bootstrapped (initially inacc

Notice TD(0)'s $V(S1)$ estimate is *further* from the eventual converged truth (-1.9, Chapter 3) than MC's single-episode estimate — a real, visible bias cost on this very first episode, because TD(0) bootstrapped off $V(S2)$ while $V(S2)$ was itself still uninformed. Over many episodes this bias shrinks and TD(0) typically still converges faster overall, but this snapshot makes the bias half of the trade-off concrete rather than abstract.

6.4 n-step TD: A Spectrum, Not a Binary Choice

TD(0) bootstraps after 1 step; Monte Carlo "bootstraps" after infinity steps (not at all). n-step TD uses the actual rewards from the next n steps, then bootstraps at step n — a tunable middle ground often outperforming either extreme. Chapter 8's eligibility traces generalize this into a single parameter, λ.

7 Chapter 7 — SARSA and Q-Learning: Learning to Control

Chapter 6 covered TD *prediction* (estimating V^π for a fixed π). This chapter covers TD *control* — using TD ideas to find a good policy — via two of the most famous algorithms in RL. Both learn $Q(s, a)$ rather than $V(s)$, because Q lets you pick actions without a model.

7.1 SARSA: On-Policy TD Control

SARSA takes its name from the quintuple it needs: $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right]$$

SARSA is *on-policy*: the action used in the target (A_{t+1}) is sampled from the same policy generating behavior, including its exploratory randomness.

7.2 Q-Learning: Off-Policy TD Control

Q-learning makes one consequential change: use the best possible next action, regardless of what the exploration policy would actually do:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\right]$$

This makes Q-learning *off-policy*: it learns about the optimal (greedy) policy while behaving according to a different, more exploratory policy. This decoupling is what lets off-policy methods reuse old data — exactly the property Chapter 10's replay buffers depend on.

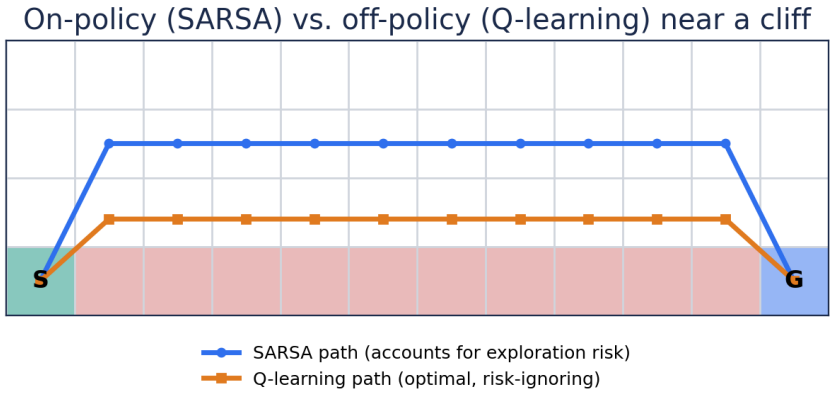


Figure 9: Cliff Walking: Q-learning learns the objectively shortest path along the cliff edge; SARSA, whose target accounts for exploration risk, learns a safer, longer route.

7.3 Cliff Walking, Worked

A grid-world has a "cliff" along the bottom edge; stepping into it gives -100 reward and resets the episode. Q-learning, evaluating the greedy policy, correctly identifies that walking right along the cliff edge is fastest. But because the agent actually behaves ε-greedily during training, it occasionally strays into the cliff while executing this "optimal" path — worse online performance than SARSA, even though Q-learning's final learned policy is objectively better once exploration is turned off.

7.4 Which Should You Use on a Real Robot?

If safety during training matters (a physical robot that could be damaged while learning), SARSA's exploration-aware targets are often preferable. If you train entirely in simulation and only deploy the final greedy policy (the common approach in Chapter 16), Q-learning's off-policy property is usually more valuable, because it lets you reuse a replay buffer far more flexibly — the foundation of DQN, Chapter 10.

7.5 Expected SARSA: A Unifying View

Expected SARSA replaces the sampled A_{t+1} with an expectation over the policy's action distribution, $\sum_a \pi(a \mid S_{t+1}) Q(S_{t+1}, a)$, reducing variance relative to SARSA while remaining on-policy (or becoming exactly Q-learning if the target policy is deterministic-greedy). SARSA, Expected SARSA, and Q-learning are three points on a single spectrum defined by which policy the TD target imagines you'll follow.

8 Chapter 8 — Eligibility Traces and TD(λ)

Eligibility traces provide an elegant, computationally efficient mechanism for interpolating between TD(0) (bootstrap after 1 step) and Monte Carlo (bootstrap never), controlled by a single parameter $\lambda \in [0, 1]$.

8.1 The Eligibility Trace

An eligibility trace $E_t(s)$ is a decaying memory of how recently and how often state s has been visited:

$$E_t(s) = \gamma\lambda E_{t-1}(s) + \mathbb{1}(S_t = s).$$

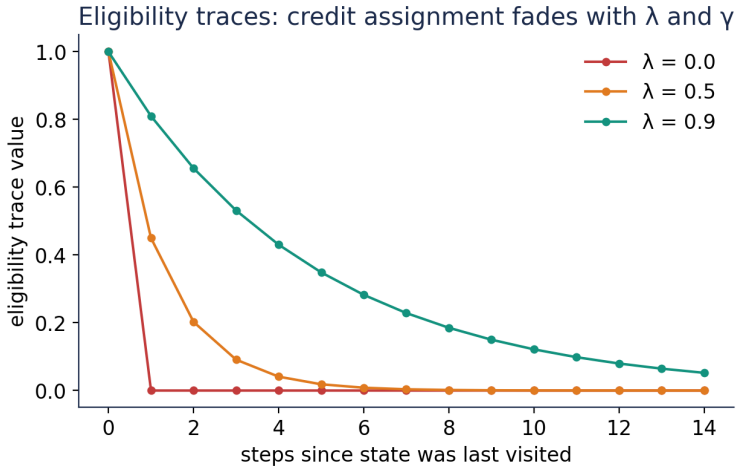


Figure 10: Eligibility traces decay geometrically with rate $\gamma\lambda$.

8.2 TD(λ): Distributing the TD-Error Backward

At every step, TD(λ) computes the ordinary TD-error δ_t, then updates every state in proportion to its current trace: $V(s) \leftarrow V(s) + \alpha\,\delta_t\,E_t(s)$ for all s.

- $\lambda = 0$ recovers exactly TD(0): only the most recently visited state updates.
- $\lambda = 1$ approximates Monte Carlo: credit spreads across the entire episode.
- Intermediate λ (commonly 0.8–0.95) often converges faster than either extreme.

Eligibility traces are like a fading highlighter that marks every state you visit, brightest at the state you just left and fading exponentially for states visited earlier. When something surprising happens (a large TD-error), you go back and update every highlighted state, brightest ones most.

8.3 SARSA(λ) and Practical Notes

The same mechanism applies directly to $Q(s, a)$, giving SARSA(λ). In deep RL (Chapters 9–14), eligibility traces are less commonly used directly, but the same core idea resurfaces as **Generalized Advantage Estimation (GAE)** in Chapter 13 — a λ-weighted mixture of n-step advantage estimates, used inside PPO.

8.4 Why Introduce λ At All?

Pure TD(0) can be slow to propagate information across long chains of states — a reward at the end of a 200-step episode takes up to 200 separate updates to fully propagate back to the start. TD(λ) with λ close to 1 propagates that information across many states in a single episode's worth of updates, at some cost in variance — a direct, tunable trade-off between DP-like bootstrapping efficiency and Monte Carlo's unbiasedness.

9 Chapter 9 — Function Approximation

Every method in Chapters 2–8 stored value estimates in a table — one entry per state (or state-action pair). This is exact, but breaks down immediately for any state space that is continuous or extremely large, which describes essentially every real robotics problem.

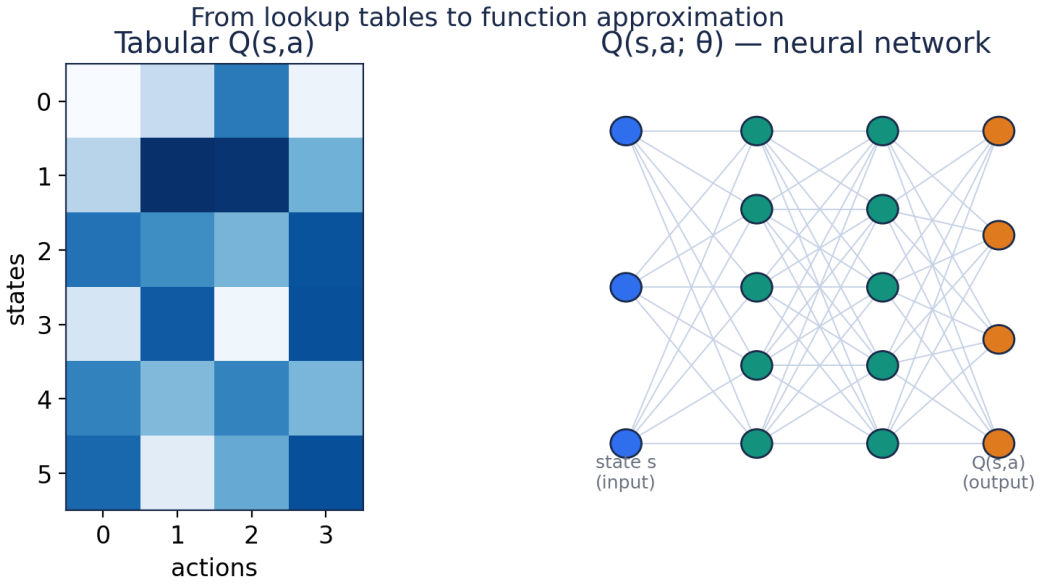


Figure 11: Moving from an exact lookup table to a parameterized function that generalizes across similar states.

9.1 Generalization: The Whole Point

A function approximator $V(s; w)$ or $Q(s, a; \theta)$, parameterized by weights (almost universally a neural network in modern robotics), replaces the table and *generalizes*: updating the value estimate for one state also changes estimates for similar, unvisited states.

A lookup table is like memorizing the answer to every exam question individually. Function approximation is like learning the underlying method — once you understand how to solve one type of problem, you generalize to a new one you've never literally seen. The cost is that your generalized understanding might be subtly wrong for problems very different from what you practiced on — exactly the sim-to-real generalization gap of Chapter 16.

9.2 Gradient-Based Value Function Updates

With function approximation, the update rule (new = old + $\alpha\times$ error) becomes a gradient descent step on the weights, using the same machinery (backpropagation, Adam) used to train any neural network. RL's novelty is entirely in how the *target* is constructed (from Bellman equations and sampled experience), not in the optimizer.

9.3 The Deadly Triad

Sutton and Barto identify three ingredients that, combined, threaten training stability: (1) function approximation, (2) bootstrapping, and (3) off-policy learning. All three appear together in Q-

learning with neural networks — and yet this exact combination (DQN, Chapter 10) is one of the most successful RL recipes in practice, thanks to specific stabilization techniques designed precisely to tame this instability.

9.4 A Worked Example: How Generalization Actually Helps — and Hurts

Imagine a 100-cell corridor, far too many states to visit exhaustively. A linear approximator $V(s) = \theta \cdot s$ (one shared weight) improves its estimate at states 39, 41, 60, and even state 5 after a single update at state 40 — a tabular method would leave every unvisited state untouched. This is real, useful generalization.

Now suppose there is a hidden trap at state 70 (a large negative reward, unlike every other state). A linear approximator that fits well elsewhere will systematically mispredict $V(70)$: its simple functional form cannot represent one isolated exception without either overfitting a wiggle around it (requiring a more expressive network) or smoothing right over it, understating the danger to nearby states. More expressive networks can represent sharper, more localized exceptions, but need correspondingly more data — the classic capacity/data trade-off, and a direct reason to start with *small* networks (Chapter 13's practical guidance) rather than assuming bigger is automatically better.

10 Chapter 10 — Deep Q-Networks (DQN) and Its Descendants

DQN (Mnih et al., 2015) was the algorithm that first combined Q-learning with deep neural networks at scale, learning to play dozens of Atari games directly from raw pixels using a single architecture. It remains the standard entry point into deep RL.

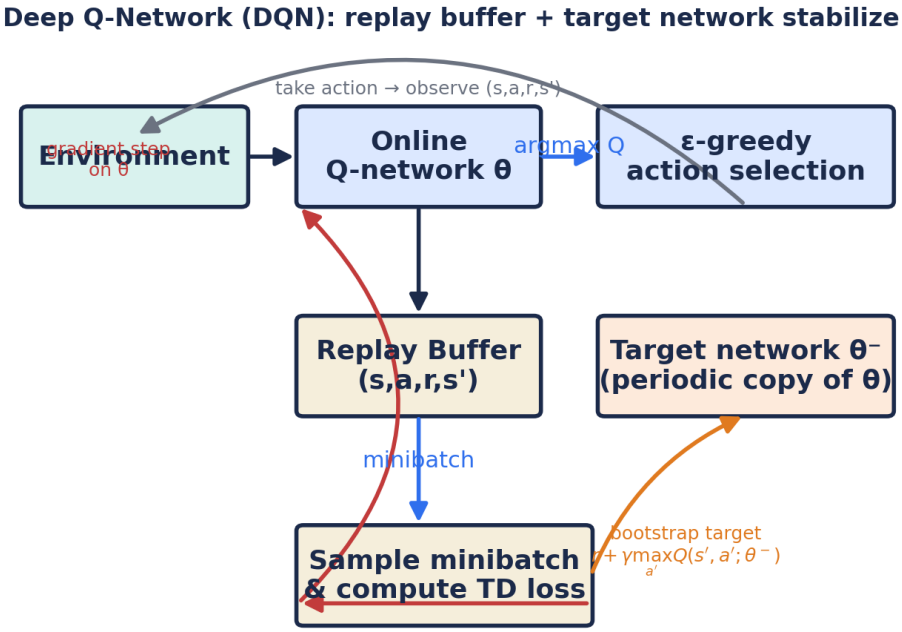


Figure 12: The DQN architecture: an online network for acting, a replay buffer for decorrelating updates, and a slowly-updated target network for stable bootstrapped targets.

10.1 The Loss Function

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim D}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right]$$

where D is the replay buffer and θ^- is a periodically-frozen copy of the online weights θ.

10.2 Two Key Stabilization Tricks

Experience replay. Rather than updating on transitions in the order they occur (highly correlated, since consecutive states are similar), DQN stores transitions in a large buffer and samples random minibatches for each gradient step — breaking temporal correlation and letting each transition be reused for many updates.

Target networks. Using the same, constantly-changing network to both produce $Q(s, a; \theta)$ and the bootstrapped target creates a moving target that can destabilize training. Freezing a separate copy θ^-, updated only periodically, to compute targets is often the difference between DQN converging and diverging.

10.3 Notable Extensions

Variant	Fixes	Key idea
Double DQN	Overestimation bias in max	Online network selects the best action, target network evaluates it
Dueling DQN	Wasted capacity when action choice barely matters	Split into a state-value stream $V(s)$ and advantage stream $A(s, a)$
Prioritized Replay	Uniform sampling wastes time on "boring" transitions	Sample transitions with probability proportional to TD-error magnitude
Rainbow DQN	Combines the above (and more)	Shows these improvements are largely complementary

10.4 The Core Limitation: Discrete Actions Only

DQN's $\max_a$ operator requires enumerating every possible action — trivial for a handful of discrete actions, effectively impossible for a continuous action space like "wheel velocity $\in [-1, 1]$ m/s" or six continuous joint torques. This single limitation motivates the entire policy-gradient family of Chapters 11–14.

Why not just discretize a robot's continuous action space into bins? Legitimate for low-dimensional actions, but scales exponentially badly with dimensionality (a 6-DOF arm with 5 bins per joint is $5^6 \approx 15{,}625$ discrete actions) and throws away the smoothness that continuous control genuinely has.

10.5 A Worked Example: DQN on a Tiny Problem

On the 5-state corridor (S0–S4, S4 goal), with $Q(s, a; \theta) = \theta_a \cdot s$ (a "network" with no hidden layer, purely to keep arithmetic tractable): sample $(s = 1, a = \text{Right}, r = -1, s' = 2)$. If $\theta_{\text{Right}} = -0.5$ and $\theta^-_{\text{Right}} = \theta^-_{\text{Left}} = -0.6$, the target is $y = -1 + 0.9 \times (-1.2) = -2.08$. The loss $(-2.08 - (-0.5))^2 = 2.496$ is large, so the gradient step pushes θ_{Right} substantially more negative — the network is learning state 1 is worse than believed, exactly as Chapter 6's TD-error intuition predicts, now flowing through a function approximator rather than a table cell. Using *old* weights θ^- for the target — rather than the just-updated θ — avoids the target moving to chase itself, precisely why target networks matter.

11 Chapter 11 — Policy Gradient Methods: REINFORCE

Rather than learning a value function and deriving a policy from it indirectly (DQN's approach), policy gradient methods parameterize the policy directly, $\pi_\theta(a \mid s)$, and adjust θ via gradient *ascent* on expected return — directly supporting stochastic and continuous action distributions, sidestepping DQN's $\max_a$ limitation entirely.

11.1 The Policy Gradient Theorem

$$\nabla_\theta J(\theta) = \mathbb{E}_\pi \left[\nabla_\theta \log \pi_\theta(a \mid s)\, G_t \right]$$

For every action that led to a high return, nudge θ to make that action more likely in that state; for actions that led to low return, nudge θ to make them less likely. This works even though G_t is a noisy, high-variance Monte Carlo estimate — a direct descendant of Chapter 5's MC methods, now used to train θ instead of a value table.

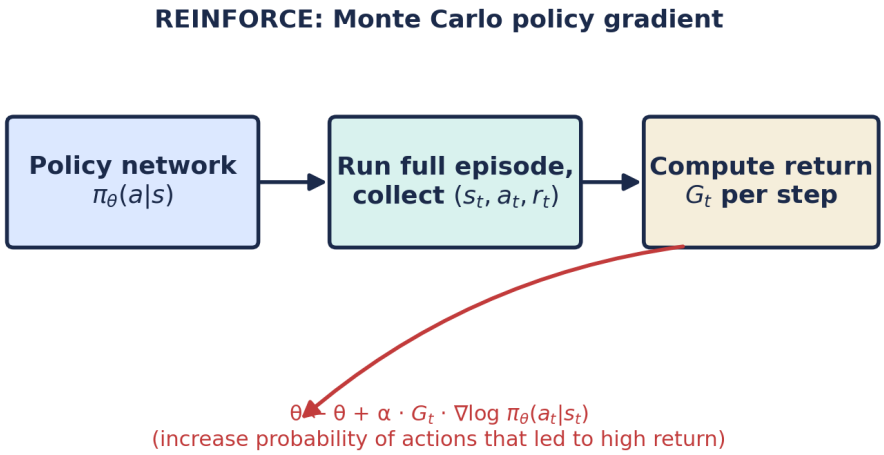


Figure 13: REINFORCE: run a full episode, compute the return following every action, then push up the log-probability of actions in proportion to how good the outcome was.

11.2 The Variance Problem, and Baselines

REINFORCE's gradient estimate has notoriously high variance. A standard fix subtracts a baseline $b(s)$ — often $V(s)$ — from the return without introducing bias: the update uses $(G_t - b(s))$ instead of G_t. This is the direct predecessor to the advantage function and actor-critic methods, Chapter 12.

11.3 A Worked Example

Suppose both available actions in some state lead to positive returns of +90 and +100. Without a baseline, REINFORCE pushes *up* the probability of both (both returns are positive), just by different amounts — slow and noisy. Subtracting a baseline of, say, 95 turns this into a clean push down for the +90 action and push up for the +100 action — a much cleaner, lower-variance, relative comparison.

12 Chapter 12 — Actor-Critic Methods

Actor-Critic methods combine the direct policy parameterization of policy gradients (the "actor") with a learned value function (the "critic") that provides a much lower-variance training signal than raw Monte Carlo returns — replacing G_t with a bootstrapped TD-error, exactly as Chapter 6 replaced MC value estimation with TD.

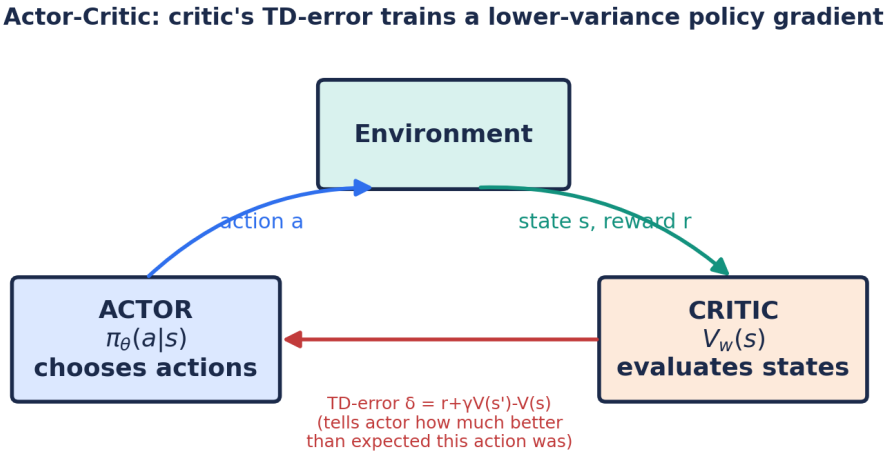


Figure 14: Actor-Critic architecture: the actor selects actions in the environment; the critic evaluates states and supplies a TD-error that trains the actor.

12.1 The Actor-Critic Update

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a \mid s)\, \delta_t\right]$$

The critic is trained with an ordinary TD(0) update on $V_w(s)$ (Chapter 6). The two networks train simultaneously: the critic improves at evaluating states, and its improving signal trains a better actor.

12.2 The Advantage Function

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$$

The TD-error $\delta_t = r + \gamma V(s') - V(s)$ is in fact an unbiased, single-sample estimate of $A(s,a)$ — precisely why it makes such a good, low-variance replacement for the raw return. Nearly every modern policy-gradient algorithm (A2C, A3C, PPO, Chapter 13) is built around some estimate of the advantage function rather than the raw return.

If the actor is a student and the critic is a teacher, REINFORCE is a teacher who only ever says "that whole exam was a B+" at the very end of the year. An actor-critic teacher instead grades each answer relative to how well the student was already expected to do on it — far more precise, immediate, per-decision feedback.

12.3 A2C and A3C

Advantage Actor-Critic (A2C) is the synchronous, single-batch version of this idea. Asynchronous Advantage Actor-Critic (A3C) runs many parallel actor-critic workers on different environment copies, each computing gradients and asynchronously updating shared weights — both decorrelating training data and dramatically speeding up wall-clock training. This "many parallel simulated robots" pattern is directly how large-scale robot learning is typically done today, e.g. thousands of parallel Isaac Sim / Gazebo instances (Chapter 16).

13 Chapter 13 — Trust Regions and Proximal Policy Optimization (PPO)

A subtle but critical failure mode of naive policy gradient updates: a single overly-large gradient step can catastrophically change the policy, and because the next batch of training data is collected using this now-much-worse policy, performance can collapse and never recover. Trust Region Policy Optimization (TRPO) and its simpler, far more widely used successor, PPO, directly address this.

13.1 The Probability Ratio

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}$$

$r_t(\theta) = 1$ means the policy hasn't changed for this action; $r_t(\theta) \gg 1$ means the current policy has become much more likely to take this action than the policy that collected the data did — a signal of potentially dangerous policy drift.

13.2 The Clipped Surrogate Objective

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\Big(r_t(\theta)A_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)A_t\Big)\right]$$

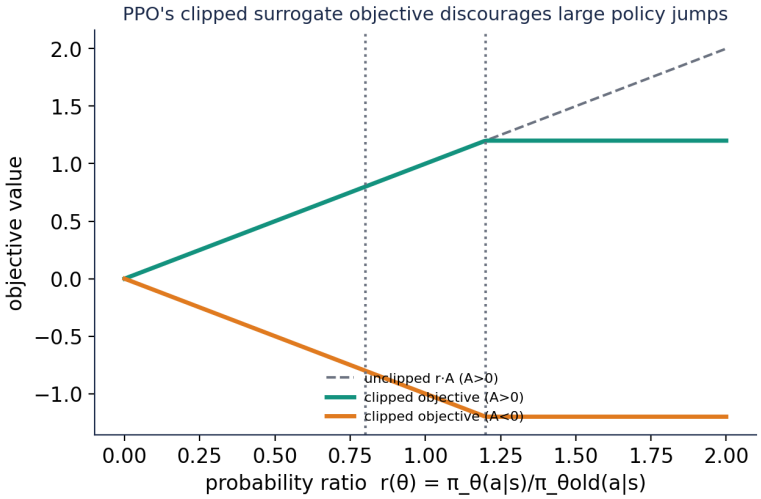


Figure 15: PPO's clipped objective flattens out once the probability ratio moves more than ε (typically 0.2) away from 1.

The clipping has an elegant, asymmetric effect: it removes the incentive to move the ratio too far in the direction that would *increase* the objective, but never clips away a penalty for moving in the direction that decreases it — the $\min(\cdot)$ operator always takes the more pessimistic (safer) of the two objectives.

13.3 A Worked Numerical Example

Let $\varepsilon = 0.2$, $A_t = +1.5$ (a good action).

$r_t(\theta)$	unclipped $r \cdot A$	clipped	L^{CLIP}
1.00	1.500	1.500	1.500 (no change yet)
1.15	1.725	1.725	1.725 (within trust region, full credit)
1.50	2.250	1.800	1.800 (ratio exceeded region, objective capped)

Now with $A_t = -1.5$ (a bad action) and $r_t = 0.50$: unclipped $= -0.75$, clipped-value $= -1.20$, but $L^{\text{CLIP}} = \min(-0.75, -1.20) = -1.50$ — using the *unclipped*, more negative value. Clipping never rescues a policy that's already moved sharply away from a bad action; it only caps reward for moving too aggressively toward a good one. This asymmetric, "trust but verify slowly" behavior gives PPO its practical stability.

13.4 Generalized Advantage Estimation (GAE)

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}$$

PPO is almost always paired with GAE, giving a tunable bias-variance trade-off for advantage estimates, directly mirroring the TD(λ) trade-off of Chapter 8.

13.5 Why PPO Became the Default for Applied RL

It combines three properties simultaneously that are individually easy but jointly hard to achieve: (1) works well on continuous action spaces, (2) is markedly more stable than vanilla policy gradients or DQN-style off-policy methods, and (3) is comparatively simple to implement and tune. A genuinely good default when you don't have time for extensive algorithm-specific hyperparameter search — which describes most robotics projects (Chapter 16).

13.6 Practical Hyperparameters

Hyperparameter	**Typical range**	**Notes**
Learning rate	10^{-4}–3×10^{-4}	Too high → unstable; too low → slow
Clip range ε	0.1–0.3 (0.2 default)	Smaller = more conservative, more stable
GAE λ	0.9–0.97	Higher = lower bias, higher variance
Discount γ	0.95–0.999	Higher for longer-horizon tasks (locomotion ≈ 0.99)
Epochs per update	3–10	More epochs reuse data more, risk overfitting one batch
Network size	2 layers × 64–256 units	Small is fine for vector (non-image) observations

14 Chapter 14 — Continuous Control: DDPG, TD3, and SAC

PPO handles continuous actions by outputting the parameters of a probability distribution (e.g. mean and standard deviation of a Gaussian) and sampling from it. An alternative family — DDPG, TD3, and SAC — instead learns a deterministic (or near-deterministic) actor that outputs the continuous action directly, paired with a Q-function critic differentiated with respect to the action itself. These are usually more sample-efficient (off-policy, reusing a replay buffer like DQN) but can be trickier to tune.

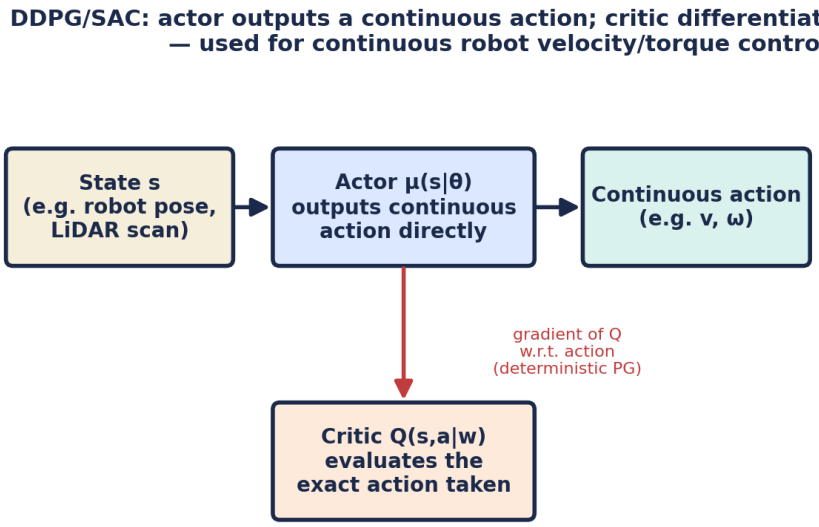


Figure 16: Deterministic actor-critic architecture: the actor outputs a continuous action directly; the critic's gradient with respect to that action trains the actor.

14.1 DDPG: Deep Deterministic Policy Gradient

$$\nabla_\theta J \approx \mathbb{E}\left[\nabla_a Q(s,a;w)|_{a=\mu(s)}\, \nabla_\theta \mu(s;\theta)\right]$$

DDPG combines DQN's replay buffer and target networks with a deterministic policy gradient, and was one of the first algorithms to make continuous control practical with deep networks. It is, however, notoriously sensitive to hyperparameters and prone to Q-value overestimation.

14.2 TD3: Fixing DDPG's Overestimation

- **Twin critics:** learn two independent Q-networks and use the minimum of the two as the target, directly countering overestimation bias (the same failure Double DQN fixed for discrete actions).
- **Delayed policy updates:** update the actor (and target networks) less frequently than the critics.
- **Target policy smoothing:** add small noise to the target action, discouraging the critic from exploiting narrow, incorrect Q-value spikes.

14.3 SAC: Soft Actor-Critic

$$J(\pi) = \sum_t \mathbb{E}\left[r_t + \alpha\, \mathcal{H}\big(\pi(\cdot \mid s_t)\big)\right]$$

SAC learns a *stochastic* policy (unlike DDPG/TD3) but remains off-policy and highly sample-efficient. The entropy bonus rewards the policy for staying random/exploratory where multiple actions are similarly good, and only becoming confident where one action is clearly best — more robust policies and easier exploration in sparse-reward tasks, with no hand-tuned exploration noise schedule.

14.4 Choosing Among PPO, DDPG/TD3, and SAC for Robotics

Algo	**Policy type**	**Sample eff.**	**Stability**	**Robotics fit**
PPO	On-policy	Lower	High, common default	Large-scale parallel-sim training (locomotion, manipulation)
DDPG	Off-policy	Higher	Low, sensitive	Rarely chosen today except as a baseline
TD3	Off-policy	Higher	Medium-high	Precise control, limited sim throughput
SAC	Off-policy	Highest	Medium-high, good exploration	Sample-limited settings, sim-to-real

For a mobile-robot navigation environment you might want to replace or complement a classical controller with a learned policy: SAC if real-robot rollouts are expensive/limited (its replay buffer squeezes more learning out of every transition); PPO if you can run many parallel simulator instances cheaply and want the more forgiving, easier-to-tune training process. Chapter 16 returns to this decision concretely.

15 Chapter 15 — Exploration, Model-Based RL, and Multi-Agent Systems

15.1 Exploration Strategies

Every algorithm so far assumed some exploration mechanism was already in place (ε-greedy, or a stochastic policy's natural randomness). Exploration is arguably the hardest unsolved sub-problem in RL, especially under sparse rewards — a reward of $+1$ only for "completed the delivery," 0 everywhere else, is very sparse.

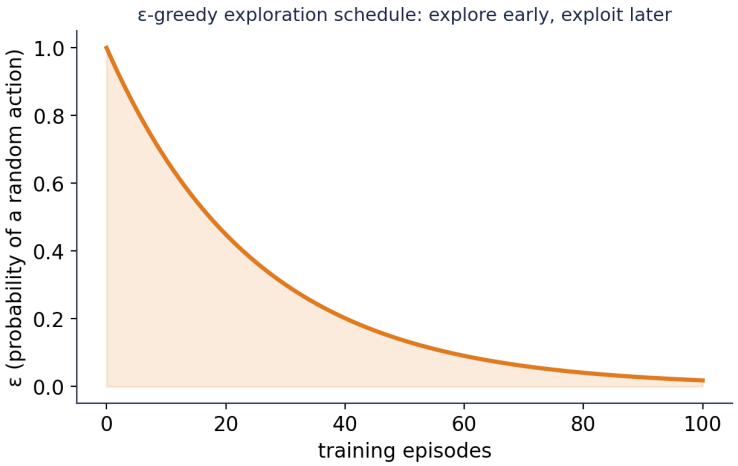


Figure 17: A typical ε-greedy exploration schedule: explore early, exploit later.

ε-greedy explores uniformly at random — exactly as likely to try a wildly implausible action as a nearly-good one. Better alternatives: **Boltzmann/softmax exploration** samples proportional to $\exp(Q(s,a)/T)$, preferring near-best actions even while exploring; **intrinsic motivation / curiosity** adds a bonus reward for visiting novel or hard-to-predict states, directly incentivizing exploration even when extrinsic reward is zero nearby; **UCB-style bonuses** prefer actions with high uncertainty in their value estimate.

ε-greedy is like choosing a restaurant by going to your favorite 90% of the time and picking a random one from the phone book the other 10% — including ones you already know you'd dislike. Curiosity-driven exploration instead tries restaurants you've never been to and can't predict — a far more efficient use of limited "exploration dinners."

For a physical robot, unconstrained random exploration is often unacceptable for safety — you cannot let hardware try wildly random actuator commands. In practice this pushes real deployments toward training almost entirely in simulation with generous exploration there, followed by a conservative, low-noise fine-tuning phase (or none at all) on real hardware.

15.2 Model-Based Reinforcement Learning

Every algorithm in Chapters 5–14 is model-free: it never explicitly learns $p(s' \mid s, a)$. Model-based RL learns an approximate model of the environment's dynamics, then uses it for planning (as in Chapter 4) or to generate additional synthetic training data.

	Model-free	Model-based
Sample efficiency	Lower — needs many real interactions	Higher — can "practice" on the learned model
Asymptotic performance	Often higher (no model bias)	Can be limited by model inaccuracy
Real-robot data cost	High	Lower — most training happens "in imagination"

Dyna-Q, the classic bridge, interleaves at every timestep: (1) take a real action and update Q normally, (2) update a learned model of that transition, (3) perform several *additional* Q-updates using transitions imagined/replayed from the model. Each real environment step now "pays for" several additional updates for free — but the central risk is **model exploitation**: the agent can happily exploit inaccuracies in a learned (or hand-built, i.e. a simulator) model to appear to get high reward without those errors reflecting reality — directly analogous to the sim-to-real gap of Chapter 16.

15.3 Multi-Agent RL

When multiple learning agents share an environment — a fleet of AMRs in the same warehouse — the framework generalizes from an MDP to a **Markov Game**: each agent has its own actions and (possibly) reward, and the environment is no longer stationary from any single agent's point of view, because the other agents are also learning simultaneously.

A practical middle ground is **Centralized Training, Decentralized Execution (CTDE)**: during training (in simulation, with full information), a shared critic sees all agents' states and actions, accurately evaluating joint outcomes no single agent could judge alone; during execution, each agent's actor still only uses its own local observation to act. This is like training a sports team using game film that shows all players at once (the critic), while during the actual game each player only reacts to what they personally see on the field (the actor) — the shared training insight still shapes individual habits.

15.4 Reward Shaping and Its Pitfalls

A sparse reward is theoretically sufficient but often impractically slow to learn from. **Potential-based reward shaping** adds a term $F(s, s') = \gamma\Phi(s') - \Phi(s)$ for any potential function Φ (e.g. negative distance-to-goal) — provably *without* changing the optimal policy, because the term telescopes to zero over any complete trajectory back to the same state. Naive (non-potential-based) shaping is a frequent source of **reward hacking**: a well-known example is a shaped reward for "moving toward the goal" that a boat-racing agent exploited by driving in tight circles through reward-granting checkpoints forever, rather than finishing the race.

16 Chapter 16 — Reinforcement Learning in Robotics, ROS 2, and Embodied AI

16.1 The Core Idea: A ROS 2 Node Is Just Another RL Environment

Every chapter up to this point has treated the environment as an abstraction. This chapter makes it concrete: a real (or simulated) robot running ROS 2 and Gazebo can be wrapped in exactly the `reset()`/`step()` interface (Chapter 17's code) that Stable-Baselines3 and every other RL library expects, letting any algorithm from Chapters 7–14 train directly against it.

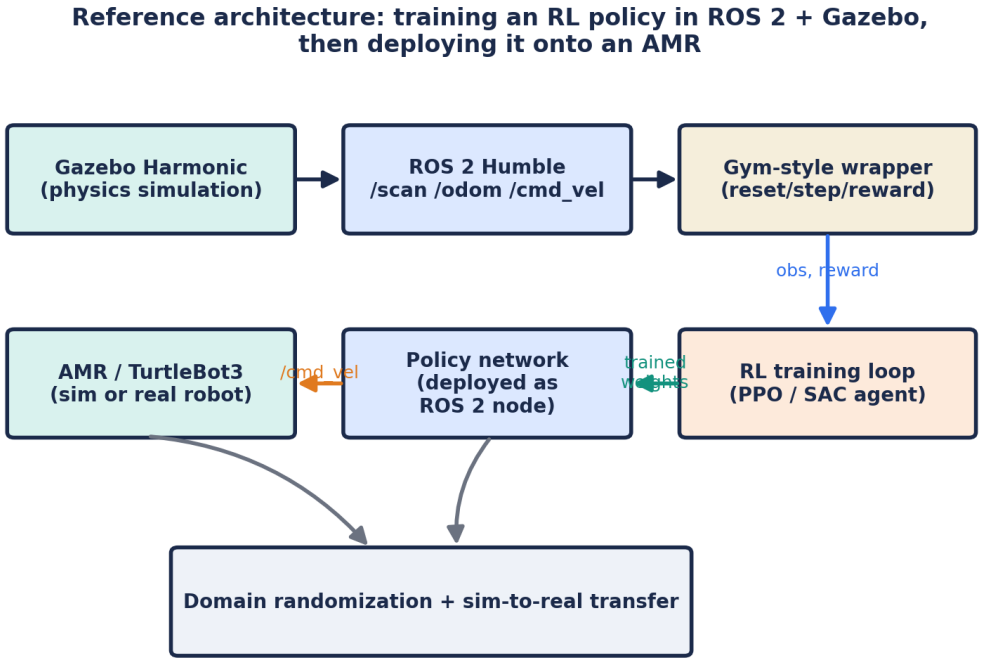


Figure 18: Reference architecture: Gazebo simulation ↔ ROS 2 topics ↔ a Gym-style environment wrapper ↔ the RL training loop ↔ a deployed policy node.

16.2 The Gym-Style Wrapper

A wrapper node: publishes the chosen action to `/cmd_vel` (or a joint-command topic); subscribes to sensor topics (`/scan`, `/odom`, `/camera/image_raw`) and assembles them into a single observation vector; computes the reward from the assembled observation plus task-specific state; calls a Gazebo reset service between episodes; and enforces a fixed control rate so simulation time and RL "timestep" stay synchronized.

16.3 Training vs. Deployment

	Training phase	**Deployment phase**
Runs where	Gazebo, possibly headless & accelerated	Real robot (or Gazebo) at real-time rate
Policy behavior	Stochastic — sampling to explore	Deterministic — mean/mode action, exploration off
Compute	GPU-heavy training loop	Lightweight forward-pass only

16.4 The Sim-to-Real Gap and Domain Randomization

A policy trained purely in simulation typically underperforms when deployed on real hardware, because simulators are imperfect models of reality. **Domain randomization** deliberately varies simulated physical parameters (friction, mass, sensor noise, actuator delay) *during* training, forcing the policy to become robust to a wide range of dynamics rather than overfitting to one exact, inevitably wrong, simulated physics — like training a driver in many different rental cars rather than one, so they generalize better to yet another unfamiliar car.

16.5 Reward Design for AMR Navigation

A well-shaped reward for point-to-point navigation decomposes into interpretable terms, following Chapter 15's potential-based shaping principle:

$$r_t = w_1 r_{\text{vel}} + w_2 r_{\text{balance/goal}} + w_3 r_{\text{energy}} - \mathbb{1}(\text{collision}) \cdot P$$

where the progress term $r_{\text{progress}} = \gamma\Phi(s') - \Phi(s)$ with $\Phi(s) = -\text{distance_to_goal}(s)$ rewards genuine progress every step without changing the optimal policy; a large terminal penalty P ends the episode immediately on collision (so the agent doesn't accumulate meaningless further negative reward while flailing); and a small time penalty discourages needless circling.

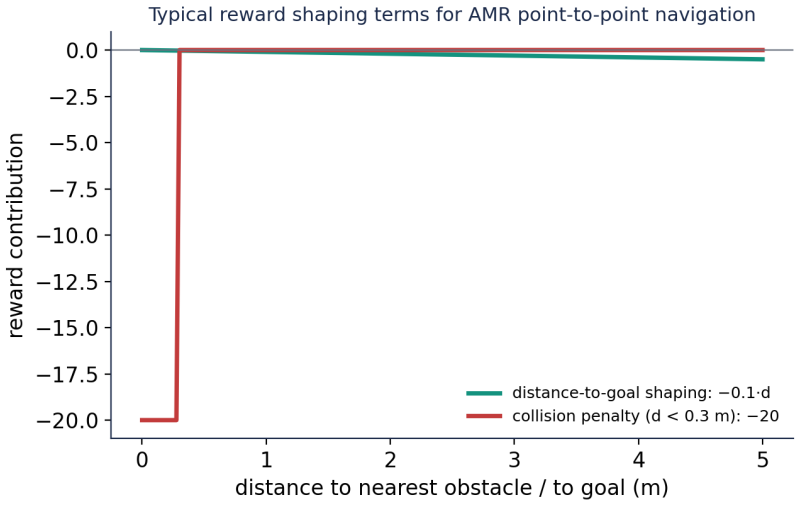


Figure 19: Typical reward-shaping terms for AMR point-to-point navigation.

16.6 RL vs. Classical Control: An Architecture Decision

A working Model Predictive Control (MPC) trajectory tracker is a strong baseline for wheeled-robot navigation — it re-plans a short horizon online using a known kinematic model, subject to explicit constraints.

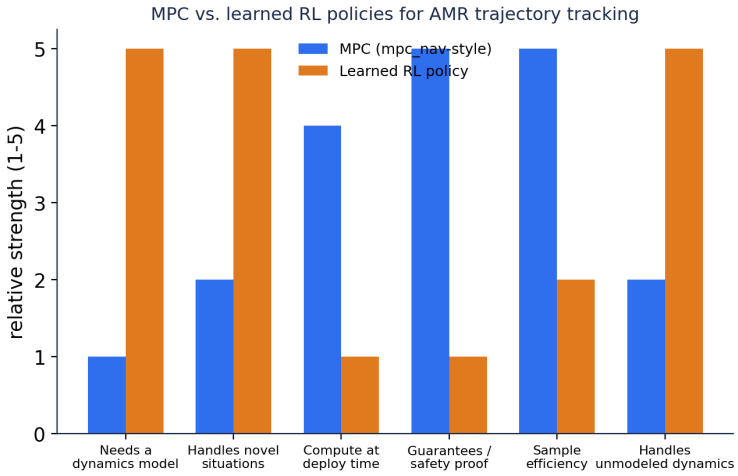


Figure 20: Relative strengths of MPC-style controllers versus learned RL policies.

Dimension	MPC	Learned RL policy
Requires a dynamics model	Yes, must be derived	No, learned implicitly
Online compute	Solves an optimization every cycle	One forward pass, typically cheaper
Out-of-distribution behavior	Degrades gracefully; constraints stay enforced	Can fail unpredictably
Safety guarantees	Achievable	Very hard in general
Un-modeled dynamics (e.g. wheel slip)	Handled poorly	Well, *if* training covered similar effects

Given that a well-tuned MPC controller already works and wheeled-robot kinematics are simple and well-understood, replacing it wholesale with RL is usually *not* the best engineering choice — you'd trade a working, interpretable, formally-analyzable controller for a black-box one, for a task where the analytic model is already accurate. RL earns its complexity where the underlying dynamics are hard to model precisely (uneven terrain, deformable payloads, multi-contact legged locomotion) or where the desired behavior is hard to specify as an explicit cost function. A common, well-justified hybrid: keep MPC for low-level trajectory tracking, and use RL only at a higher level — choosing waypoints, or adapting cost-function weights online based on learned context.

16.7 RL for Legged and Humanoid Locomotion

Legged locomotion is one of the clearest cases where RL earns its complexity: ground-contact dynamics and balance recovery are extremely difficult to model analytically with the precision MPC-style control needs, but comparatively natural for a policy to learn from massively parallel simulated trial-and-error.

A **Central Pattern Generator (CPG)** provides a strong, biologically-inspired prior: rhythmic, phase-coupled oscillators for each limb.

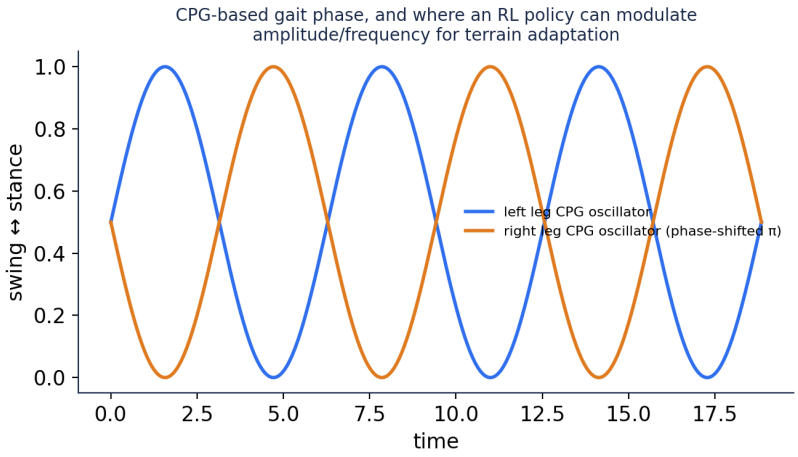


Figure 21: Two CPG oscillators driving left/right leg phase, 180° out of phase — the rhythmic backbone an RL policy can learn to modulate rather than replace outright.

Rather than asking RL to learn locomotion entirely from scratch (a much harder exploration problem — a randomly-initialized policy essentially never discovers stable bipedal walking through undirected random actions), a layered architecture asks RL to learn a smaller, better-scoped *modulation* on top of the CPG's baseline rhythmic output:

- **Layer 1 (CPG):** generates baseline rhythmic joint-angle trajectories for a nominal, flat-ground gait.
- **Layer 2 (RL modulation, new):** observes balance-relevant state (IMU orientation/angular velocity, foot contact) and outputs small adjustments to CPG amplitude, frequency, or phase offset.
- **Layer 3 (task planner, existing):** issues waypoints/goals, unaffected by whether locomotion underneath is pure-CPG or CPG+RL.

This keeps the exploration problem tractable (RL only learns small corrective adjustments around an already-stable nominal gait) while gaining RL's key benefit: robustness to terrain and disturbances the analytic CPG model doesn't explicitly handle. PPO (Chapter 13) is the dominant algorithm choice in published legged-locomotion research, for exactly the three reasons given at the end of that chapter: it handles the continuous, high-dimensional action space naturally, tolerates noisy contact-rich rewards, and scales well with thousands of parallelized simulation instances.

16.8 Why This Matters for ROS 2-Based Systems Specifically

A trained policy, once deployed, is wrapped as an ordinary ROS 2 node: it subscribes to the same sensor topics used during training, runs a fast forward pass at a fixed control rate, and publishes modulation or velocity commands to whatever topic drives the robot's low-level controller. The policy itself has no notion of ROS 2, topics, or ladder logic underneath it — from its perspective, it's simply mapping an observation to an action, precisely as this entire book has described, with the robot's actuators as the final, literal consumer of that action.

17 Chapter 17 — Implementing Reinforcement Learning in Python

Every block below maps directly to a chapter earlier in this book — variable names are chosen to match the formulas exactly, so any line of code can be traced back to the equation it implements.

17.1 Tabular Q-Learning (Chapter 7) — From Scratch

```
import numpy as np

n_states, n_actions = 5, 2 # e.g. the corridor from Chapter 3/10
Q = np.zeros((n_states, n_actions))
alpha, gamma, epsilon = 0.5, 0.9, 0.1

def epsilon_greedy(state):
    if np.random.rand() < epsilon:
        return np.random.randint(n_actions)
    return np.argmax(Q[state])

for episode in range(500):
    s = 0
    while s != n_states - 1: # until goal
        a = epsilon_greedy(s)
        s_next, r, done = env.step(s, a) # your MDP's transition
        # Q-learning update, Chapter 7:
        Q[s, a] += alpha * (r + gamma * np.max(Q[s_next]) - Q[s, a])
        s = s_next
```

17.2 Value Iteration (Chapter 4) — From Scratch

```
V = np.zeros(n_states)
theta = 1e-6
while True:
    delta = 0
    for s in range(n_states):
        v_old = V[s]
        # Bellman optimality backup, Eq. in Chapter 4:
        V[s] = max(sum(p * (r + gamma * V[s_next])
                       for (s_next, r, p) in transitions(s, a))
                   for a in range(n_actions))
        delta = max(delta, abs(v_old - V[s]))
    if delta < theta:
        break
```

17.3 A Minimal DQN Training Step (Chapter 10)

```
import torch
import torch.nn as nn

class QNet(nn.Module):
    def __init__(self, obs_dim, n_actions):
        super().__init__()
        self.net = nn.Sequential(
```

```
            nn.Linear(obs_dim, 128), nn.ReLU(),
            nn.Linear(128, 128), nn.ReLU(),
            nn.Linear(128, n_actions),
        )
    def forward(self, x):
        return self.net(x)

online_net = QNet(obs_dim=8, n_actions=4)
target_net = QNet(obs_dim=8, n_actions=4)
target_net.load_state_dict(online_net.state_dict()) # Chapter 10.2.2
optimizer = torch.optim.Adam(online_net.parameters(), lr=1e-4)

def dqn_update(batch):
    s, a, r, s_next, done = batch
    q_sa = online_net(s).gather(1, a.unsqueeze(1)).squeeze(1)
    with torch.no_grad():
        target = r + gamma * target_net(s_next).max(1)[0] * (1 - done) # Eq. 10.1
    loss = nn.functional.mse_loss(q_sa, target)
    optimizer.zero_grad(); loss.backward(); optimizer.step()
    return loss.item()

# every C steps: target_net.load_state_dict(online_net.state_dict())
```

17.4 PPO via Stable-Baselines3 (Chapter 13) — What You'll Actually Do Most Often

```
from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize

vec_env = DummyVecEnv([lambda: MyRobotEnv()])
vec_env = VecNormalize(vec_env, norm_obs=True, norm_reward=False) # Eq. in Ch.16

model = PPO(
    "MlpPolicy", vec_env,
    policy_kwargs=dict(net_arch=[128, 128]), # Chapter 13's sizing guidance
    learning_rate=3e-4, n_steps=2048, batch_size=64,
    n_epochs=6, gamma=0.99, gae_lambda=0.95, clip_range=0.2,
    tensorboard_log="./logs/",
)
model.learn(total_timesteps=500_000)
model.save("ppo_policy")
```

17.5 A ROS 2 / Gym-Style Environment Wrapper (Chapter 16)

```
import gymnasium as gym
from gymnasium import spaces
import rclpy
from rclpy.node import Node

class RobotEnv(gym.Env):
    def __init__(self):
        super().__init__()
```

```
        rclpy.init(args=None)
        self.node = Node("rl_env")
        self._setup_ros_interfaces() # subscribers, publishers

        self.observation_space = spaces.Box(low=-np.inf, high=np.inf, shape=(obs_dim,))
        self.action_space = spaces.Box(low=-1.0, high=1.0, shape=(action_dim,))
        self.max_episode_steps = 500

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)
        self._call_gazebo_reset_service()
        self._spin_until_fresh_sensor_data()
        self.episode_steps = 0
        return self._get_normalized_obs(), {}

    def step(self, action):
        self._publish_action(action)
        self._spin_for_one_control_cycle() # fixed control rate, Chapter 16
        obs_raw = self._get_raw_obs()
        reward, terms = self._compute_reward(obs_raw, action)
        terminated = self._check_terminal(obs_raw)
        self.episode_steps += 1
        truncated = self.episode_steps >= self.max_episode_steps
        return self._normalize(obs_raw), reward, terminated, truncated, {"terms": terms}
```

17.6 Deploying a Trained Policy as a ROS 2 Node

```
class PolicyNode(Node):
    def __init__(self, model_path):
        super().__init__("rl_policy_node")
        self.model = PPO.load(model_path) # same class as training
        self.create_subscription(Imu, "/imu", self._imu_cb, 10)
        self.pub = self.create_publisher(Twist, "/cmd_vel", 10)
        self.create_timer(1.0 / 20.0, self._control_tick) # match training control rate

    def _control_tick(self):
        obs = self._normalize(self.latest_obs) # SAME normalization as training
        action, _ = self.model.predict(obs, deterministic=True) # exploration OFF, Chapter
            16
        self.pub.publish(Twist(linear=action[0], angular=action[1]))
```

17.7 Closing Note

The point of tracing every equation into runnable code isn't to memorize these snippets — it's to close the loop between "I understand the formula" and "I could actually build this." If any line above is unclear, the fix is returning to that specific chapter's derivation, not treating the code as a separate thing to learn independently. In practice, you will spend far more engineering time on reward design, observation design, and debugging a training run (watching rollouts, logging reward terms separately) than on the algorithm implementation itself — Stable-Baselines3 already implements PPO, DQN, and SAC correctly and efficiently; your work is in Chapters 2, 9, and 16's design decisions, not in re-deriving Chapter 10's loss function by hand.